LAB 07 - PROMPTS

Signal Stage: a Library of Trend, Mean-Reversion, Momentum and Breakout Signals

Open your AI coding assistant (for example Claude Code or GitHub Copilot) in the labs_fullcode folder. Work down the steps: copy each prompt into the assistant, review the code it writes, then run the command. The complete, working code is already in Lab07-technical-indicators/solution/ if you want to compare. Never paste API keys into an AI chat.

Step 1: Agree the signal contract (A2)

Do this (no prompt needed)
Read the docstring of the reference Lab07-technical-indicators/solution/signals.py. Every signal returns a float Series of target positions (1 = long, 0 = flat) decided with information up to the CLOSE of bar t; the position is HELD from bar t+1. Warm-up bars (NaN indicators) are flat.

Check: You can explain why a signal computed at Friday's close can only earn Monday's return.

Step 2: Build the trend signals (A2, K3)

Prompt - copy into your AI coding assistant
Create labs_fullcode/pipeline/signals.py. Helper _flat_until_ready(cond, *inputs) returns 1.0 where cond is true and 0.0 otherwise or while any input is NaN. Trend signals using pipeline.indicators: ema_cross(bars, fast=10, slow=30) long while ta_ema(fast) > ta_ema(slow); sma_cross(bars, fast=50, slow=200); trend_filter(bars, window=200) long while close > ta_sma(window). Each returns a float Series aligned to bars.index, using data up to bar t only. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Check: ema_cross(load_bars('SPY','2018-01-01','2025-12-31')).iloc[-1] is 1.0.

Step 3: Build mean-reversion, momentum and breakout signals (A2, K3)

Prompt - copy into your AI coding assistant
In labs_fullcode/pipeline/signals.py add enter_exit(entry, exit_) - a stateful long/flat series that is 1 from an entry bar until the next exit bar (entry wins a tie, NaN treated as False); rsi_reversion(bars, n=2, entry=10, exit=70) = enter_exit(RSI < entry, RSI > exit); dip_buy(bars, n_down=3, hold=3) = long for `hold` bars after a bar completing >= n_down consecutive down closes; momentum(bars, lookback=126, threshold=0.0) long while the past lookback-bar return > threshold; donchian_breakout(bars, entry=55, exit=20) buy a close above the PRIOR entry-bar high (rolling max shifted 1), sell a close below the prior exit-bar low; bollinger_breakout(bars, window=20, k=2.0) buy a close above the upper band, sell below the middle band. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Check: Each function returns only 0.0 and 1.0 values for SPY.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Add the registry, next-bar alignment and the look-ahead test (A2, K3)

Prompt - copy into your AI coding assistant
In labs_fullcode/pipeline/signals.py add SIGNALS = {name: (function, family, default_params)} for the eight signals; target_position(name, bars, **params) (params override defaults, Series renamed to name); next_bar_position(target) = target.shift(1).fillna(0) (the position actually held); lookahead_test(name, bars, checks=25, seed=0, fn=None, **params): pick 25 random dates t in the last two-thirds of the sample and recompute the signal (a) on bars.loc[:t] only and (b) after multiplying every price AFTER t by 1.5 and by 0.5; count dates where any value up to t differs from the full-sample signal (any difference = look-ahead); describe_signal(target) with latest_target, entries, exposure, avg_hold_bars and last_change; latest_targets(symbols, name, params, start, end) returning a DataFrame of symbol, as_of, close, target, last_change. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Check: lookahead_test('ema_cross', bars) returns 0.

Step 5: Build the signals command (A2)

Prompt - copy into your AI coding assistant
In labs_fullcode/pipeline/signals.py add run(symbol='SPY') for `python -m pipeline signals --symbol`: for every signal in SIGNALS print family, params, latest target, entries, exposure, average holding bars, last change date and the look-ahead result (PASS or FAIL (n)); print one example of next-bar alignment for ema_cross (the close where the target changed and the first bar it is held); compute latest_targets for config universe.trade with config strategy name/params, print it and save _outputs/Lab07_targets.csv (input to the execution stage); save chart Lab07_signals.png (SPY 2024-2025 with green shading where ema_cross, rsi_reversion, momentum and donchian_breakout are long). Exit code 1 if any look-ahead test fails. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Check: python -m pipeline signals --help shows --symbol.

Step 6: Run the signal stage (A2, K3)

Do this (no prompt needed)
Run: python -m pipeline signals --symbol SPY (same as: python3 Lab07-technical-indicators/signals_stage.py).

Check: ema_cross 26 entries, 71% exposure, 54.7 bars average hold, last change 2025-11-28; sma_cross 5 entries; trend_filter 24; rsi_reversion 88 entries, 20% exposure, 4.6 bars; dip_buy 102 entries, 19%, 3.7 bars; momentum 33; donchian_breakout 20; bollinger_breakout 37 (latest target 0); all look-ahead PASS. Alignment: target changes to 1 at the close of 2025-11-28 and is held from 2025-12-01. Universe: 7 of 9 symbols long (MSFT and TLT flat); _outputs/Lab07_targets.csv and Lab07_signals.png.

Step 7: Catch a look-ahead bug on purpose (K3)

Do this (no prompt needed)
Temporarily change ema_cross to use bars['close'].shift(-1) (tomorrow's close) and rerun the stage. Then undo the change and rerun.

Check: With the bug ema_cross shows 'FAIL (25)' (the future-shock test changes its history every time) and the command exits with code 1; after undoing it all signals PASS again.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 8: Relate signal frequency to trading cost (K7)

Prompt - copy into your AI coding assistant
Using the signal table from python -m pipeline signals, estimate for each signal the round trips per year over 2018-2025 and the annual cost drag at 3 bps per round trip on USD 100,000, assuming the full equity is traded. Rank the signals from most to least cost-sensitive and say which ones need a strong edge to survive costs.

Open in Claude | **Open in ChatGPT**

Check: dip_buy and rsi_reversion trade about 11-13 round trips a year versus under one for sma_cross, so they are the most cost-sensitive.

Run it

Step	Command / file
Run Lab 07	python3 Lab07-technical-indicators/signals_stage.py (same as: python -m pipeline signals)
The complete code	pipeline/signals.py (reference copy: Lab07-technical-indicators/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.